

ab_rollback

ab_rollback.sh — user guide

Revision: 1 **Last modified:** 2026-06-11T10:30:00Z

Companion documentation (§11.4.18) for tests/emulator/ab_virt/
ab_rollback.sh.

Overview

ab_rollback.sh proves the **real, bootloader-driven A/B auto-rollback** of the RK3588 A/B-virt emulator on an Apple-Silicon host: it boots the SAME 2-slot GPT disk (ab_disk.img) with the SAME real u-boot.bin and drives the U-Boot bootcount/bootlimit guard so that a slot whose boot attempts exceed bootlimit is rolled back to the **previous-good** slot by the in-script boot.cmd rollback guard (BOOT_ORDER swap), and asserts that the guest userspace observably reports the rolled-back slot.

- **Run ROLLBACK** — BOOT_ORDER="B A", bootcount=2, bootlimit=1 (so bootcount > bootlimit) → boot.cmd step-2 guard fires: 'rolling back (altbootcmd swap)' → BOOT_ORDER swaps to head A → active_slot=A → root=/dev/vda2 → guest /etc/slot_id=A.
- **Run CONTROL** — BOOT_ORDER="B A", bootcount=1, bootlimit=1 (so bootcount is NOT over bootlimit) → NO rollback → head stays B → active_slot=B → guest /etc/slot_id=B.

The proof is the **metamorphic contrast** (§11.4.107): the ONLY difference between the two runs is the bootcount value relative to bootlimit. A fake / no-op rollback guard would boot the SAME slot in both runs (RED); the real guard rolls back to A only when the counter guard trips, and leaves B alone when it does not (GREEN). Both consoles are captured in full — real getty login + post-login command output, a live shell, not a frozen frame (§11.4.107).

This is the U-Boot bootcount auto-rollback primitive (the Mender/RAUC bootcount / bootlimit / altbootcmd convention) running under a real **U-Boot 2024.01 + QEMU -machine virt + HVF** — not a mock. It is the auto-rollback phase that ab_slot_switch.sh (the deliberate slot switch, PWU-AB-1) is the counterpart of.

Proven status (FACT, §11.4.6): this auto-rollback has run GREEN. Evidence: docs/qa/20260611T095918Z-ab-rollback/ — verdict.txt records the Run ROLLBACK guard firing (U-Boot 'rolling back (altbootcmd swap)',

active_slot=A, root=/dev/vda2, guest slot_id=A) under BOOT_ORDER='B A' bootcount=2 bootlimit=1, and the Run CONTROL (bootcount=1, bootlimit=1) booting slot_id=B with NO rollback; real u-boot.bin: U-Boot 2024.01, Verdict: PASS; full transcripts in the captured consoles.

What is NOT yet proven (§11.4.6 — honest boundary): this test proves the **in-RAM-session** bootcount guard ONLY — bootcount and bootlimit are set interactively at the U-Boot prompt within a single boot session, and the guard correctly rolls the head slot back to the previous-good slot when bootcount > bootlimit. It does NOT prove persistent saveenv power-cycle rollback (where U-Boot itself increments + persists bootcount across real reboots before boot.scr runs); that is a later PWU. It does NOT prove RAUC bundle apply / dm-verity / AVB-style slot integrity (PWU-AB-2). No claim is made here that those work.

Prerequisites

- A completed build_image.sh run: out/.ok present and out/images/u-boot.bin non-empty (the script aborts with exit 3 otherwise).
- A completed assemble_ab_disk.sh run: out/.disk_ok present and out/images/ab_disk.img non-empty (exit 3 otherwise).
- qemu-system-aarch64 on PATH with HVF support.
- expect on PATH.
- HELIX_AB_ROOT_PW must match the password baked into the image by build_image.sh (default helixota).

Usage examples

```
# Run the full A/B auto-rollback proof (after build_image.sh +  
    assemble_ab_disk.sh  
# have produced GREEN out/.ok + out/.disk_ok).  
tests/emulator/ab_virt/ab_rollback.sh  
  
# With a non-default root password (must match the built image).  
HELIX_AB_ROOT_PW=helixota tests/emulator/ab_virt/ab_rollback.sh
```

Inputs

Input	Source	Default / requirement
HELIX_AB_ROOT_PW	env → ROOT_PW	helixota
out/.ok, out/images/ u-boot.bin	filesystem (from build_image.sh)	required, non-empty
out/.disk_ok, out/ images/ab_disk.img	filesystem (from assemble_ab_disk.sh)	required, non-empty

The run id is derived from a UTC timestamp: <YYYYmmddTHHMMSSZ>-ab-rollback.

Outputs

All under docs/qa/<run-id>/ (the captured evidence, §11.4.83):

Output	Meaning
docs/qa/<run-id>/ console transcript(s)	full QEMU transcripts of the ROLLBACK run and the CONTROL run
docs/qa/<run-id>/ driver log(s)	the expect driver's own stdout/stderr per run
docs/qa/<run-id>/verdict.txt	distilled verdict: u-boot version, per-run bootcount/bootlimit/BOOT_ORDER, the rollback-guard line, the per-run active_slot + rootdev + guest slot_id, Verdict: PASS/FAIL
stdout PASS/FAIL lines	one [PASS]/[FAIL] per anti-bluff assertion + a final RESULT line
exit code	0 = PASS, 1 = FAIL, 3 = precondition abort

Side-effects

- Spawns two transient qemu-system-aarch64 guests (the ROLLBACK run and the CONTROL run); each exits on guest poweroff or the expect timeout (self-cleaning, §11.4.14).
- Creates the docs/qa/<run-id>/ evidence directory.
- The rollback guard's BOOT_ORDER swap + saveenv happen inside the transient guest's in-RAM U-Boot env session; the runs set bootcount/bootlimit/BOOT_ORDER interactively per run, so the on-disk default env is exercised within a single session rather than persisted across a host-driven power-cycle.

Internal behaviour

1. set -u + set -o pipefail; resolve SCRIPT_DIR, repo root, root password, u-boot.bin/ab_disk.img paths, run id, and the evidence dir; mkdir -p the evidence dir.
2. **Preconditions** (each → exit 3 on failure): out/.ok present; out/.disk_ok present; u-boot.bin non-empty; ab_disk.img non-empty; qemu-system-aarch64 found; expect found.
3. Emit a deterministic expect driver via a single-quoted heredoc — so there is exactly ONE quoting level (Tcl) and guest-shell command substitution \\$(...) reaches the file verbatim, ensuring the GUEST shell (not Tcl) runs it (§11.4.1 — the FAIL must be a product defect, never a script bug).
4. For each run (ROLLBACK then CONTROL) the driver:
 - spawns QEMU -M virt -accel hvf -cpu host -nographic -bios <u-boot.bin> -drive file=<ab_disk.img>,if=virtio,format=raw;
 - interrupts U-Boot autoboot, reaches the => U-Boot prompt;
 - sets the run's env: setenv BOOT_ORDER "B A", setenv bootcount <2|1>, setenv bootlimit 1;

- `load virtio 0:1 ... boot.scr then source ...` (runs `uboot_ab/boot.cmd`, whose step-2 guard rolls `BOOT_ORDER` back to head A and prints rolling back (`altbootcmd swap`) WHEN `bootcount > bootlimit`, then selects the head slot and boots the kernel with `root=/dev/vda<part> + helix_slot=<slot>`);
 - waits for `buildroot login:`, logs in as `root`;
 - from inside the guest shell prints `/etc/slot_id`, the mounted root device, `/proc/cmdline`, and a post-login sentinel; then `poweroff`.
 - the transcript is captured per run via `log_file`.
5. **Anti-bluff assertions** — `grep` the captured consoles; the proof is the metamorphic contrast:
- Run `ROLLBACK (bootcount=2 > bootlimit=1)` must show the rolling back (`altbootcmd swap`) U-Boot line, `active_slot=A`, `root=/dev/vda2`, guest `/etc/slot_id=A`, `helix_slot=A`, and a live post-login sentinel — the head slot was rolled back from B to A.
 - Run `CONTROL (bootcount=1 == bootlimit=1, NOT over)` must show NO rollback line, `active_slot=B`, `root=/dev/vda3`, guest `/etc/slot_id=B` — head slot B was kept (the rollback is real, not a no-op that always swaps).
6. Write `verdict.txt` (u-boot version, per-run env + guard line + slot, Verdict: PASS/FAIL). If every assertion holds, print `RESULT: PASS` + the evidence path and exit 0; otherwise print `RESULT: FAIL` (with the `qemu/expect rc`'s) and exit 1.

Edge cases

- **out/.ok / out/.disk_ok absent, or u-boot.bin / ab_disk.img missing** → exit 3 (run `build_image.sh` then `assemble_ab_disk.sh` first).
- **QEMU or expect not installed** → exit 3.
- **No autoboot / U-Boot prompt / boot.scr load / login / password / shell prompt within timeout** → the driver emits a driver-fail line and exits non-zero; the corresponding contrast assertions then FAIL and the script exits 1.
- **A fake rollback guard that ALWAYS swaps (ignores bootcount)** → the CONTROL run would also boot slot A → the CONTROL negative assertion (`/etc/slot_id=B`, NO rollback line) FAILs → Verdict: FAIL, exit 1.
- **A fake guard that NEVER swaps** → the ROLLBACK run would still boot slot B → its positive assertions FAIL → Verdict: FAIL, exit 1.
- **Poweroff times out** → treated as a non-fatal note; the verdict still depends solely on the captured-evidence contrast assertions.

Related scripts

- `tests/emulator/ab_virt/ab_slot_switch.sh` — the deliberate bootloader-driven slot switch (PWU-AB-1); the rollback here is the bootcount-driven counterpart. See `docs/scripts/ab_slot_switch.md`.
- `tests/emulator/ab_virt/assemble_ab_disk.sh` — produces the 2-slot GPT `ab_disk.img` + compiled `boot.scr` this script boots. See `docs/scripts/assemble_ab_disk.md`.
- `tests/emulator/ab_virt/build_image.sh` — produces the kernel Image
 - `rootfs.ext2` + `u-boot.bin`. See `docs/scripts/build_image.md`.

- `tests/emulator/ab_virt/boot_smoke.sh` — base-image boot smoke (the proven foundation, single-slot). See `docs/scripts/boot_smoke.md`.
- `uboot_ab/boot.cmd` + `uboot_ab/README.md` — the U-Boot A/B boot script source (compiled to `boot.scr`) this test sources; its step-2 guard is the rollback engine under test, and its `bootcount/bootlimit/altbootcmd` contract is what this test drives.
- `docs/research/rk3588_emulator/REPORT.md` — the emulator design report.

Last verified date

2026-06-11 — documented against the script as committed (read, not inferred). The bootloader-driven A/B auto-rollback is PROVEN GREEN; evidence at `docs/qa/20260611T095918Z-ab-rollback/verdict.txt` (Run `ROLLBACK BOOT_ORDER='B A' bootcount=2 bootlimit=1` → rolling back (`altbootcmd swap`) → active slot A / `vda2` / guest `slot_id=A`; Run `CONTROL bootcount=1` → guest `slot_id=B`, no rollback; U-Boot 2024.01). Persistent `saveenv` power-cycle rollback and RAUC `dm-verity` remain UNPROVEN (later PWUs).